

```

// EMERGENCE OS 1 - Master Engine
// Full title: [REBUILT] EMERGENCE OS 1 - Independent NPCs, Trust & Grudges,
// and Auto-Updating Location Cards
//
// Feature set:
//   - Human Agency: NPCs resist unearned demands, hold cognitive biases, and
//     shift through polyvagal threat states (Composed/Fight/Flight/Freeze/Fawn).
//   - Persistent Trust & Grudge tracking, Ego Depletion, and physical micro-tell
//   - Undercurrents: NPC-to-NPC relationship dynamics inferred PASSIVELY from
//     what's already happening in the story - no scheduled "start a storyline"
//     events, no cooldown/dormancy lifecycle, no hidden write-back tag asking
//     the narrator to report developments. It just reads what actually
//     happened (reusing the same trigger patterns that drive player-NPC
//     trust/grudge) and lets each NPC's own attachment style color how that
//     reads in the text. Nothing is pre-scheduled; nothing is invented.
//   - Reflections: opt-in, per-NPC private first-person thoughts saved to their
//     own story card, captured via a hidden tag (the same mechanism used for
//     character profile fill, for internal consistency). Never enter story
//     context, never affect behavior.
//   - AI-authored character profiles: new /card sheets get filled in by the
//     narrator itself (accurately if it recognizes the character, invented but
//     fitting if it's original) instead of sitting on blank placeholders.
//   - Dynamic World Tension, Player Trauma, and multi-location damage/condition
//     tracking that keeps each place's state independent of the others.
//   - Auto-Genre Adaptation and slash commands for direct state inspection
//     (/npc, /npcs, /undercurrents, /romance, /reflections, /settings, /world,
//     /loc, /card, /cleanup, /forget) that never waste a story turn.
//
// Built to work within AI Dungeon's real context-priority system: Front Memory
// (system state) is kept deliberately tiny since AI Dungeon never trims it, while
// heavier/variable content (NPC color, nudges) goes through Author's Note, which
// AI Dungeon can gracefully shrink under pressure. Story cards are created via
// AI Dungeon's own addStoryCard() API rather than raw array pushes.

if (typeof state === 'undefined') {
  state = {};
}

class EmergenceEngine {
  static init() {
    if (!state.emergence) state.emergence = {};
    if (!state.emergence.config) state.emergence.config = {};

    const defaultCfg = {

```

```

Genre: "Modern", AutonomyLevel: "High", PsychologicalRealism: "Raw Human",
HumanAgency: "Enabled", GrudgeTracking: "Enabled", WorldTensionEngine: "Dyn
PlayerTrauma: "Enabled", PhysicalQuirks: "High", RomanceEngine: "18+ Unrest
JealousyMechanic: "Enabled", MatureContent: "18+ Unrestricted", GraphicReal
LocationCards: "Enabled", LocationAutoUpdate: "Enabled", InnerSelfSystem: "
LivingWorldEngine: "Enabled", ProtagonistInvolvement: "Medium", Consequence
DialogueStyle: "Raw & Gritty", NPCBrainSystem: "Enabled",
ReflectionSystem: "Disabled", ReflectionInterval: "6", ReflectionChance: "5
NPCColorNotes: "Enabled"
};

for (let key in defaultCfg) {
  if (state.emergence.config[key] === undefined) {
    state.emergence.config[key] = defaultCfg[key];
  }
}

if (!state.emergence.genreScores) {
  state.emergence.genreScores = {
    "Superhero": 0, "Sci-Fi": 0, "Horror": 0, "Fantasy": 0,
    "Western": 0, "Cyberpunk": 0, "Noir": 0, "Modern": 0
  };
}

if (!state.emergence.activeThreads) state.emergence.activeThreads = [];
if (state.emergence.worldTension === undefined) state.emergence.worldTension
if (!state.emergence.currentLocation) state.emergence.currentLocation = "Unkn
if (state.emergence.turnCount === undefined) state.emergence.turnCount = 0;
if (!state.emergence.reflectionBook) {
  state.emergence.reflectionBook = { lastTurn: 0, seq: {} };
}
if (!state.emergence.sceneNames) state.emergence.sceneNames = [];
if (!state.emergence.nameCandidates) state.emergence.nameCandidates = {};
// All three of these are queues, not single values - a second /card, a
// second romance stage change, or a second Earned Security shift in the
// same turn used to silently overwrite and abandon whichever one fired
// first. Migrates gracefully from the old single-value shapes.
if (!Array.isArray(state.emergence.pendingProfileFill)) {
  const old = state.emergence.pendingProfileFill;
  state.emergence.pendingProfileFill = (old && old.name) ? [old] : [];
}
if (!Array.isArray(state.emergence.pendingRomanceStageChange)) {
  const old = state.emergence.pendingRomanceStageChange;
  state.emergence.pendingRomanceStageChange = (old && old.name) ? [old] : [];
}
if (!Array.isArray(state.emergence.pendingSecurityShift)) {
  const old = state.emergence.pendingSecurityShift;

```

```

    state.emergence.pendingSecurityShift = old ? [old] : [];
}

if (!state.world) state.world = {};
if (!state.world.npcs) state.world.npcs = {};
if (!state.world.locations) state.world.locations = {};
if (!state.playerInner) state.playerInner = { stress: 15, egoReserve: 100, co

Object.keys(state.world.npcs).forEach(k => {
    let npc = state.world.npcs[k];
    if (!npc || typeof npc !== 'object') {
        npc = {};
        state.world.npcs[k] = npc;
    }
    if (!npc.memories) npc.memories = [];
    if (npc.coreMemory === undefined) npc.coreMemory = "";
    if (!npc.hiddenGoal) npc.hiddenGoal = "Survive.";
    if (!npc.secret) npc.secret = "None.";
    if (!npc.coreBelief) npc.coreBelief = "Unsure of their place.";
    if (!npc.threatState) npc.threatState = "Composed";
    if (!npc.outerMask) npc.outerMask = "Guarded.";
    if (!npc.innerMind) npc.innerMind = "Observing.";
    if (npc.trust === undefined) npc.trust = 50;
    if (npc.grudge === undefined) npc.grudge = 0;
    if (npc.stress === undefined) npc.stress = 10;
    if (npc.egoReserve === undefined) npc.egoReserve = 100;
    if (npc.attraction === undefined) npc.attraction = 0;
    if (!npc.cognitiveBias) npc.cognitiveBias = "Skeptical";
    if (!npc.attachmentStyle) npc.attachmentStyle = "Secure";
    if (!npc.mood) npc.mood = "Neutral";
    if (!npc.undercurrents || typeof npc.undercurrents !== 'object' || Array.is
        // Migrate from the old single-target shape if present, otherwise start f
        if (npc.undercurrent && npc.undercurrent.target && npc.undercurrent.warmt
            npc.undercurrents = { [npc.undercurrent.target]: npc.undercurrent.warmt
        } else {
            npc.undercurrents = {};
        }
    }
    delete npc.undercurrent;
    if (!Array.isArray(npc.reflections)) npc.reflections = [];
    if (!npc.lastSeenLocation) npc.lastSeenLocation = "";
    if (npc.lastSeenTurn === undefined) npc.lastSeenTurn = 0;
    if (npc.secureStreak === undefined) npc.secureStreak = 0;
});
}

// Prefer AI Dungeon's own action counter (info.actionCount) over a

```

```

// self-incremented one: a self-incremented counter drifts out of sync if the
// player edits or retries an action, since Context still runs again. Falls
// back to self-increment if info isn't available in this sandbox.
static syncTurnCount() {
    if (typeof info !== 'undefined' && info && typeof info.actionCount === 'numbe
        state.emergence.turnCount = info.actionCount;
    } else {
        state.emergence.turnCount = (state.emergence.turnCount || 0) + 1;
    }
    return state.emergence.turnCount;
}

// AutonomyLevel scales how hard NPCs push back against unearned demands.
static autonomyMultiplier() {
    const map = { "Low": 0.5, "Medium": 1, "High": 1.5, "Unchained": 2.2 };
    const v = map[state.emergence.config.AutonomyLevel];
    return v !== undefined ? v : 1;
}

// ConsequenceSeverity scales how hard betrayal/lying lands.
static severityMultiplier() {
    const map = { "Mild": 0.5, "Moderate": 1, "Hardcore": 1.6 };
    const v = map[state.emergence.config.ConsequenceSeverity];
    return v !== undefined ? v : 1;
}

static clamp(val, min = 0, max = 100) {
    return Math.max(min, Math.min(max, val));
}

static escapeRegExp(str) {
    return String(str).replace(/[\.*+?^${}()|[\]\$\&']/g, '\\$&');
}

static toTitleCase(str) {
    return String(str).replace(/\w\S*/g, (txt) => txt.charAt(0).toUpperCase() + t
}

// Case-insensitive NPC lookup, used by nearly every command and the tag
// write-back handlers - one place instead of the same find() repeated at
// every call site. Returns the NPC's actual stored key (correct casing) or
// null.
static findNPCByName(name) {
    if (!name) return null;
    const target = String(name).toLowerCase();
    return Object.keys(state.world.npcs).find(n => n.toLowerCase() === target) ||
}

```

```

// A short, concrete excerpt of the actual moment, so a memory reads as
// something that really happened rather than a generic category label.
static excerptFrom(text, maxLen = 60) {
  if (!text) return "";
  let t = String(text).trim().replace(/\s+/g, ' ');
  if (t.length <= maxLen) return t;
  let cut = t.slice(0, maxLen);
  const lastSpace = cut.lastIndexOf(' ');
  if (lastSpace > 20) cut = cut.slice(0, lastSpace);
  return cut + "...";
}

static updateStat(obj, stat, change, min = 0, max = 100) {
  if (!obj) return;
  if (obj[stat] === undefined) obj[stat] = 50;
  obj[stat] = this.clamp(obj[stat] + change, min, max);
}

// Centralized card creation through AI Dungeon's own addStoryCard() function
// (which assigns a proper id, matching how the platform's UI creates cards)
// instead of pushing a raw object onto storyCards directly. Falls back to a
// manual push (still with an id, for consistency) if addStoryCard isn't
// available in this sandbox for any reason.
static createStoryCard(keys, entry, type, name, notes) {
  if (typeof addStoryCard === 'function') {
    try {
      const card = addStoryCard(keys, entry, type, name, notes, { returnCard: true });
      if (card && typeof card === 'object') return card;
    } catch (e) {}
  }
  if (typeof storyCards === 'undefined' || !Array.isArray(storyCards)) return null;
  const card = {
    id: Math.floor(Math.random() * 10000000000).toString(),
    title: name, name: name, keys: keys, entry: entry, description: notes || ""
  };
  storyCards.push(card);
  return card;
}

// Cached, richly-conjugated keyword patterns for behavior/condition detection.
// Plain \bWORD\b patterns only match the bare dictionary form - "demands",
// "attacked", "kissed" etc. never match \bdemand\b / \battack\b / \bkiss\b -
// which silently missed most real English usage. Built once and reused.
static getTriggerPatterns() {
  if (!this._triggerPatterns) {
    const build = (words) => new RegExp(`\\b(${words.join('|')})\\b`, 'i');

```

```

    this._triggerPatterns = {
      tensionUp: build(["explosions?", "gunfire", "shout(?:s|ed|ing)?", "scream
      traumaUp: build(["damages?", "damaged", "damaging", "injur(?:y|ies|ed)",
      traumaDown: build(["rests?", "rested", "resting", "safe(?:ty)?", "sanctua
      pushback: build(["demands?", "demanded", "demanding", "orders?", "ordered
      respect: build(["asks?", "asked", "asking", "please", "helps?", "helped",
      betrayal: build(["lies?", "lied", "lying", "betrays?", "betrayed", "betra
      romance: build(["blush(?:es|ed|ing)?", "kiss(?:es|ed|ing)?", "flirts?", "
      locFire: build(["fires?", "burn(?:ing|ed|t)?", "flames?", "flaming", "bla
      locDestroyed: build(["explosions?", "explod(?:ed|ing|es)", "destroy(?:ed|
      locLocked: build(["locks?", "locked", "locking", "seals?", "sealed", "sea
      locSafe: build(["safe(?:ty)?", "secure[ds]?", "securing", "repair(?:ed|in
    };
  }
  return this._triggerPatterns;
}

// Packs candidate Author's Note items into a character budget by priority
// (lowest number first) instead of naive concatenate-then-truncate, so a
// time-limited item (like a profile-fill request with only 2 attempts left)
// can't get silently crowded out by a routine one that happened to come first.
static packByPriority(candidates, budget) {
  const usable = candidates.filter(c => c && c.text);
  usable.sort((a, b) => a.priority - b.priority);
  let notes = "";
  usable.forEach((c, i) => {
    if (i === 0) {
      // The highest-priority candidate always gets included, truncated if
      // it's larger than the whole budget by itself - previously, an
      // oversized top candidate got silently dropped entirely rather than
      // ever appearing at all, which is exactly what was happening to the
      // 831-character profile-fill request against a 700-char budget.
      notes = c.text.length <= budget ? c.text : c.text.slice(0, budget - 3) +
      return;
    }
    const merged = notes + '\n\n' + c.text;
    if (merged.length <= budget) {
      notes = merged;
    }
    // else: skip this one (over budget) but keep trying lower-priority
    // items in case a shorter one still fits in the remaining space.
  });
  return notes;
}

static ensureConfigCard() {
  this.init();
}

```

```
    if (typeof storyCards === 'undefined' || !Array.isArray(storyCards)) return;
    try {
        const exists = storyCards.some(c => c && (String(c.keys).toLowerCase()).incl
        if (!exists) {
            const notesStr = `This card is EMERGENCE OS's control panel. You don't ne
below to use the script - it's already running. Everything here is optional
fine-tuning for when you want it. Two things worth knowing up front:
1) NPCs are detected automatically as they appear in your story - you don't
    need to add them yourself. The character list below is just a shortcut.
2) Type /help as an action anytime to see what you can check on (their
    feelings, relationships, memories, etc.) without it costing you a turn.
```

[CHARACTERS]

```
# List character first names below, one per line, to manually integrate them
# with the script - this gives them trust/grudge/bias/threat tracking right
# away instead of waiting for the script to auto-detect them from the story.
# Example:
# Marcus
# Elena
```

[SETTINGS GUIDE]

Grouped by what each setting actually affects, with what you'd actually notice in play, not just the raw option list. Edit the WORLD CONFIG block above using these exact values. Everything defaults to a fully-on, realistic setup - you only need to touch any of this if you want something different.

-- STORY TONE --

Genre: auto-detected as you play, or set it yourself (Modern, Fantasy, Sci-Fi, Ho
DialogueStyle: free text - the tone shown directly to the narrator (e.g. "Raw & G

-- NPC PSYCHOLOGY --

NPCBrainSystem: Enabled | Disabled - master switch for the whole per-NPC system:
AutonomyLevel: Low | Medium | High | Unchained - how hard NPCs push back against
HumanAgency: Enabled | Disabled - lets NPCs resist, doubt, or walk away instead o
PsychologicalRealism: Standard | Advanced | Raw Human - Raw Human prompts subcons
PhysicalQuirks: High | Moderate | Off - how often the narrator is prompted to wea
NpcColorNotes: Enabled | Disabled - the compact per-turn NPC status note (threat
InnerSelfSystem: Enabled | Disabled - whether each NPC's hidden Goal/Secret/Belie

-- TRUST, GRUDGE & CONSEQUENCES --

GrudgeTracking: Enabled | Disabled - persistent trust/grudge memory of how you tr
ConsequenceSeverity: Mild | Moderate | Hardcore - how hard betrayal and lying spe

-- RELATIONSHIPS BETWEEN NPCs --

LivingWorldEngine: Enabled | Disabled - master switch for everything NPC-to-NPC:
JealousyMechanic: Enabled | Disabled - whether public romance with one NPC visibl
RomanceEngine: 18+ Unrestricted | PG-13 | Disabled - master switch for attraction

RomancePacing: Fast | Normal | Slow Burn - how quickly attraction and relationshi

-- WORLD & LOCATIONS --

WorldTensionEngine: Dynamic | Static | Disabled - whether gunfire/explosions/chao

PlayerTrauma: Enabled | Disabled - whether high player stress injects sensory-dis

LocationCards: Enabled | Disabled - master switch for auto-tracked location Story

LocationAutoUpdate: Enabled | Disabled - whether existing location cards update a

-- PRIVATE REFLECTIONS --

ReflectionSystem: Enabled | Disabled - off by default. Lets NPCs occasionally for

ReflectionInterval: minimum turns between reflection attempts (default 6).

ReflectionChance: percent chance per eligible turn once the interval has passed (

-- PACING --

ProtagonistInvolvement: Low | Medium | High | Always - how much NPC-to-NPC gossip

-- CONTENT --

MatureContent: 18+ Unrestricted | Disabled - explicit, unfiltered narrative permi

GraphicRealism: Unfiltered | Standard | Mild - combat and violence detail level.

Not sure where to start? Everything above is already on and working - you don't need to change anything. The two settings people adjust most are AutonomyLevel (if pushback feels too soft or too harsh) and RomancePacing (if relationships are moving faster or slower than you'd like).

Type /help for the full command list, or /settings for a shorter version of this

```
    this.createStoryCard(  
      "master_world_config",  
      `[WORLD CONFIG]\nGenre: Modern\nInnerSelfSystem: Enabled\nLivingWorldEn  
      "Custom",  
      "Master World Config",  
      notesStr  
    );  
  }  
} catch (e) {}  
}
```

```
static syncSettings() {  
  if (typeof storyCards === 'undefined' || !Array.isArray(storyCards)) return;  
  try {  
    const card = storyCards.find(c => c && (String(c.keys).toLowerCase().includ  
    if (card) {  
      const lines = (card.entry || "").split('\n');  
      lines.forEach(line => {  
        const match = line.match(/^[A-Za-z]+:\s*(.+)$/);  
        if (match && state.emergence.config[match[1]] !== undefined) {
```



```

        state.emergence.config[match[1]] = match[2].trim();
    }
});

const notesToScan = card.description || card.notes || "";
const charsMatch = notesToScan.match(/\[CHARACTERS\]([\s\S]*?)\[SETTINGS
if (charsMatch) {
    const charLines = charsMatch[1].split('\n');
    charLines.forEach(line => {
        const name = line.trim();
        // Skip blank lines and comment lines (anything starting with #) -
        // only bare, uncommented lines are treated as real character names.
        if (!name || name.startsWith('#')) return;
        if (!state.world.npcs[name]) {
            this.initializeNPC(this.toTitleCase(name));
        }
    });
}
} catch (e) {}
}

static processCommand(cmdText) {
    const cmd = cmdText.trim().toLowerCase();

    if (cmd.startsWith('/help')) {
        return "🌀 EMERGENCE OS is already running in the background - you don't ne
    }

    if (cmd.startsWith('/settings')) {
        return "🔧 SETTINGS (edit on the Master World Config card):\nNPCBrainSystem
    }

    if (cmd.startsWith('/loc ')) {
        const locName = this.toTitleCase(cmdText.substring(5).trim());
        state.emergence.currentLocation = locName;
        this.processLocationCards(true);
        return `📍 Location updated to: ${locName}. Smart Location Card generated/s
    }

    if (cmd.startsWith('/cleanup')) {
        const originalCount = Object.keys(state.world.npcs).length;
        const ignore = ["Dan", "Eve", "Door", "Table", "Shadows", "Sun", "Moon", "C
        const locNames = Object.keys(state.world.locations).map(l => l.toLowerCase(
        Object.keys(state.world.npcs).forEach(n => {
            if (ignore.includes(n) || locNames.includes(n.toLowerCase())) delete stat
        });
    }

```

```

    return `🧹 Cleanup complete. Removed ${originalCount - Object.keys(state.world.npcs)}
  }

  if (cmd.startsWith('/forget ')) {
    const name = cmdText.substring(8).trim();
    const actualName = this.findNPCByName(name);
    if (!actualName) return `🤖 NPC '${name}' not found. Try /npcs for a list.`
    delete state.world.npcs[actualName];
    if (typeof storyCards !== 'undefined' && Array.isArray(storyCards)) {
      const reflectKey = "eos-reflect:" + actualName.toLowerCase();
      for (let i = storyCards.length - 1; i >= 0; i--) {
        const c = storyCards[i];
        if (c && (c.title === actualName || c.keys === reflectKey)) storyCards[i] = null;
      }
    }
    return `🧹 Forgot ${actualName} - removed from tracking along with any relationships
  }

  if (cmd.startsWith('/undercurrents') || cmd.startsWith('/drives') || cmd.startsWith('/emergence')) {
    const names = Object.keys(state.world.npcs);
    const seen = {};
    const lines = [];
    names.forEach(n => {
      const npc = state.world.npcs[n];
      if (!npc.undercurrents) return;
      Object.keys(npc.undercurrents).forEach(target => {
        const pairKey = [n, target].sort().join('|');
        if (seen[pairKey]) return;
        seen[pairKey] = true;
        const warmth = npc.undercurrents[target];
        const tone = warmth > 0 ? "warmth" : "tension";
        lines.push(`• ${n} ↔ ${target}: ${tone} (${warmth})`);
      });
    });
    let out = "";
    if (lines.length) {
      out += "🌊 Undercurrents:\n" + lines.join('\n');
    } else {
      out += "🌊 No undercurrents have formed between NPCs yet - they emerge from the void\n";
    }
    if (state.emergence.activeThreads.length) {
      out += "\n\n🗣️ Recent Gossip:\n" + state.emergence.activeThreads.map(t => t.text).join('\n');
    }
    return out;
  }

  if (cmd.startsWith('/factions')) {

```

```

const groups = this.detectCoalitions();
if (!groups.coalitions.length && !groups.rivalryBlocs.length) {
  return "👊 No emergent factions yet - a Coalition or Rivalry Bloc forms w
}
const avgOf = (members, stat) => Math.round(members.reduce((sum, n) => sum
let out = "";
if (groups.coalitions.length) {
  out += "👊 Coalitions:\n" + groups.coalitions.map(c => `• ${c.join(', ')}
}
if (groups.rivalryBlocs.length) {
  if (out) out += "\n\n";
  out += "👊 Rivalry Blocs:\n" + groups.rivalryBlocs.map(c => `• ${c.join('
}
return out;
}

if (cmd.startsWith('/reflections') || cmd.startsWith('/thoughts')) {
  const name = cmdText.replace(/^\/(reflections|thoughts)/, '').trim();
  if (!name) return "🗨 Usage: /reflections [name]";
  const actualName = this.findNPCByName(name);
  if (!actualName) return `🗨 NPC '${name}' not found. Try /npcs for a list.`
  if (state.emergence.config.ReflectionSystem !== "Enabled") {
    return "🗨 Reflections are currently disabled. Set ReflectionSystem: Enab
  }
  const npc = state.world.npcs[actualName];
  if (!npc.reflections || !npc.reflections.length) return `🗨 No reflections
  return `🗨 ${actualName}'s Private Reflections:\n` + npc.reflections.map(r
}

if (cmd.startsWith('/card ')) {
  const name = cmdText.substring(6).trim();
  let actualName = this.findNPCByName(name);

  if (!actualName) {
    actualName = this.toTitleCase(name);
    this.initializeNPC(actualName);
  }
  return this.generateCharacterCard(actualName);
}

if (cmd.startsWith('/npcs')) {
  const names = Object.keys(state.world.npcs);
  if (names.length === 0) return "🗨 No NPCs discovered yet.";
  return "🗨 Active NPCs:\n" + names.map(n => `• ${n} | Trust: ${state.world.
}

if (cmd.startsWith('/locations')) {

```

```

const names = Object.keys(state.world.locations);
if (names.length === 0) return "🗺️ No locations tracked yet - these fill in
return "🗺️ Known Locations:\n" + names.map(n => {
    const loc = state.world.locations[n];
    const marker = n === state.emergence.currentLocation ? " (current)" : "";
    return `• ${n}${marker} | ${loc.condition} | ${this.locationAtmosphere(lo
    }).join('\n');
}

if (cmd.startsWith('/npc ')) {
    const name = cmdText.substring(5).trim();
    const actualName = this.findNPCByName(name);
    if (!actualName) return `🤖 NPC '${name}' not found. Try /npcs for a list.`

    const npc = state.world.npcs[actualName];
    let out = `👤 DOSSIER: ${actualName}\n`;
    out += `❤️ Trust: ${npc.trust}/100 | 😡 Grudge: ${npc.grudge}/100\n`;
    out += `⚠️ Threat: ${npc.threatState} | 🧠 Bias: ${npc.cognitiveBias} | 🧬
    if (npc.lastSeenLocation) {
        const turnsAgo = Math.max(0, (state.emergence.turnCount || 0) - (npc.last
        out += `📍 Last seen: ${npc.lastSeenLocation}${turnsAgo > 0 ? ` (${turnsA
    }
    if (state.emergence.config.InnerSelfSystem === "Enabled") {
        out += `🎯 Hidden Goal: ${npc.hiddenGoal}\n🤫 Secret: ${npc.secret}\n🕒 1
    }
    out += `🎭 Outer Mask: ${npc.outerMask}\n🧠 Inner Mind: ${npc.innerMind}\n`
    if (state.emergence.config.RomanceEngine !== "Disabled" && npc.attraction >
        out += `💕 Romance: ${this.getRelationshipStage(npc.attraction)} (${npc.a
    }
    const strongest = this.strongestUndercurrent(npc);
    if (strongest) {
        const tone = strongest.warmth > 0 ? "warmth" : "tension";
        out += `🌊 Undercurrent: ${tone} with ${strongest.target} (${strongest.wa
    }
    if (npc.reflections && npc.reflections.length) {
        out += `🌀 Reflections logged: ${npc.reflections.length} (use /reflection
    }
    if (npc.attachmentStyle !== "Secure" && npc.secureStreak > 0) {
        out += `🔒 Building earned security: ${npc.secureStreak}/10 (sustained tr
    }
    out += `🧠 Core Memory: ${npc.memories.length > 0 ? npc.memories[npc.memori
    if (npc.coreMemory) out += `\n⭐ Formative Memory: ${npc.coreMemory}`;
    return out;
}

if (cmd.startsWith('/romance')) {
    if (state.emergence.config.RomanceEngine === "Disabled") {

```

```

    return "💕 Romance Engine is currently disabled. Set RomanceEngine to a v
}
const name = cmdText.substring(8).trim();
if (!name) {
    const names = Object.keys(state.world.npcs).filter(n => state.world.npcs[
    if (!names.length) return "💕 No romantic feelings have developed yet.";
    return "💕 Romantic Standing:\n" + names.map(n => {
        const npc = state.world.npcs[n];
        return `• ${n}: ${this.getRelationshipStage(npc.attraction)} (${npc.att
    }).join('\n');
    }
    const actualName = this.findNPCByName(name);
    if (!actualName) return `🤖 NPC '${name}' not found. Try /npcs for a list.`
    const npc = state.world.npcs[actualName];
    return `💕 ${actualName}: ${this.getRelationshipStage(npc.attraction)} (${n
}

if (cmd.startsWith('/reputation')) {
    const names = Object.keys(state.world.npcs);
    if (!names.length) return "🇺🇸 No NPCs discovered yet to have a reputation w
    const avgTrust = Math.round(names.reduce((sum, n) => sum + state.world.npcs
    const avgGrudge = Math.round(names.reduce((sum, n) => sum + state.world.npc
    let standing;
    if (avgTrust >= 70 && avgGrudge <= 20) standing = "Well-regarded";
    else if (avgGrudge >= 60) standing = "Widely distrusted";
    else if (avgTrust <= 30) standing = "Not well known or liked";
    else standing = "Mixed reputation";
    return `🇺🇸 OVERALL REPUTATION (across ${names.length} known NPC${names.leng
}

if (cmd.startsWith('/world')) {
    const loc = state.world.locations[state.emergence.currentLocation];
    let locLine;
    if (state.emergence.currentLocation === "Unknown Location") {
        locLine = "Not yet established - this fills in automatically once the sto
    } else {
        const locCond = loc ? loc.condition : "Intact";
        const atmosphere = loc ? this.locationAtmosphere(loc) : "Calm";
        locLine = `${state.emergence.currentLocation} (${locCond}, ${atmosphere})
    }
    return `🌍 WORLD STATE:\n📍 Location: ${locLine}\n🔥 Tension: ${state.emerg
}

return null;
}

static generateCharacterCard(name) {

```

```

    if (typeof storyCards === 'undefined') return "⚠️ Story Cards unavailable.";
    if (!state.world.npcs[name]) {
        this.initializeNPC(name);
    }

    const exists = storyCards.find(c => c && c.title && c.title.toLowerCase() === name);
    if (exists) return "⚠️ Card for ${name} already exists.`;

    const template = `Name: ${name}\nRace: [Define Race]\nStrength Level: [Define
    const card = this.createStoryCard(name.toLowerCase(), template.substring(0, 1
    if (card) card.category = "Characters";

    this.updateCharacterCards();
    state.emergence.pendingProfileFill.push({ name: name, attempts: 0 });
    return `✅ Character Card created for ${name} in 'Characters'. Filling in the
}

static updateCharacterCards() {
    if (typeof storyCards === 'undefined') return;
    storyCards.forEach(card => {
        if (card && card.category === "Characters" && state.world.npcs[card.title])
            const npc = state.world.npcs[card.title];
            let mem = npc.memories.length > 0 ? npc.memories[npc.memories.length - 1]
            let stateBlock = `--- EMERGENCE STATE ---\nTrust: ${npc.trust}/100 | Grud

        if (state.emergence.config.InnerSelfSystem === "Enabled") {
            stateBlock += `\nGoal: ${npc.hiddenGoal}\nBelief: ${npc.coreBelief}`;
        }
        const strongestUC = this.strongestUndercurrent(npc);
        if (strongestUC) {
            stateBlock += `\nUndercurrent: ${strongestUC.warmth > 0 ? "warmth" : "t
        }
        if (npc.coreMemory) {
            stateBlock += `\nFormative Memory: ${npc.coreMemory}`;
        }

        if (card.entry) {
            let baseLore = card.entry.split('--- EMERGENCE STATE ---')[0].trim();
            let newEntry = `${baseLore}\n\n${stateBlock}`;
            if (newEntry.length > 1500) {
                newEntry = newEntry.substring(0, 1497) + "...";
            }
            card.entry = newEntry;
        }
    }
}
});

```

```

}

static discoverLocations(text) {
  if (!text || state.emergence.config.LocationCards !== "Enabled") return;
  const invalidLocs = ["Door", "Table", "Room", "House", "City", "Street", "Car",
    "Shadows", "Sun", "Moon", "He", "She", "It", "They", "We", "His", "Her", "Y",
    "Their", "Its", "Him", "Them", "This", "That", "These", "Those", "Something",
    "Everyone", "Everything", "Nothing", "Anybody", "Anyone"];

  // Three complementary patterns, since a place can be introduced several ways
  //   "you enter the Cafe"           - preposition + the/a/an + Name
  //   "the city of Larion"           - place-noun + of + Name (name comes AFTER)
  //   "Larion City" / "Neo Academy" - Name + place-noun (name comes BEFORE)
  const patterns = [
    /\b(?:in|into|at|inside|enters?|entering|approaching|towards|arriving at|re
    /\b(?:[Cc]ity|[Tt]own|[Vv]illage|[Kk]ingdom|[Rr]ealm|[Ll]and|[Nn]ation|[Rr]
    /\b([A-Z][a-z]+(?:\s[A-Z][a-z]+)?) (?:City|Town|Village|Kingdom|Realm|Acade
  ];

  patterns.forEach(re => {
    re.lastIndex = 0;
    let match;
    while ((match = re.exec(text)) !== null) {
      let potentialLoc = match[1].trim();
      if (potentialLoc && !invalidLocs.includes(potentialLoc) && !state.world.n
        if (!state.world.locations[potentialLoc] && Object.keys(state.world.loc
          return; // cap reached - keep tracking known locations, stop register
        }
        state.emergence.currentLocation = potentialLoc;
        if (state.emergence.config.LocationAutoUpdate === "Enabled") {
          this.processLocationCards(false);
        }
      }
    }
  });
}

static updateLocationConditions(text) {
  if (!text) return;

  const patterns = this.getTriggerPatterns();
  const conditionOf = (chunk) => {
    if (patterns.locFire.test(chunk)) return "On Fire";
    if (patterns.locDestroyed.test(chunk)) return "Destroyed";
    if (patterns.locLocked.test(chunk)) return "Locked Down";
    if (patterns.locSafe.test(chunk)) return "Intact";
    return null;
  };
}

```

```

};

// Split into rough clauses on sentence breaks and contrast words ("but",
// "while", etc.) so "X is destroyed, but Y is safe" attributes each
// condition to the right place instead of blending both together.
const marked = text
  .replace(/([.!?;])\s+/g, '$1\u0001')
  .replace(/\s+\b(but|while|whereas|yet|however|although)\b\s+/gi, '\u0001$1')
const clauses = marked.split('\u0001');

const known = Object.keys(state.world.locations);
let matchedAny = false;
known.forEach(locName => {
  const re = new RegExp(`\\b${this.escapeRegExp(locName)}\\b`, 'i');
  const ownClauses = clauses.filter(c => re.test(c));
  if (!ownClauses.length) return;
  matchedAny = true;
  const newCond = conditionOf(ownClauses.join(' '));
  if (newCond && newCond !== state.world.locations[locName].condition) {
    state.world.locations[locName].condition = newCond;
    if (state.emergence.config.LocationAutoUpdate === "Enabled") this.process
  }
});

if (!matchedAny) {
  const locName = state.emergence.currentLocation;
  if (locName === "Unknown Location") return;
  if (!state.world.locations[locName]) state.world.locations[locName] = { con
  const newCond = conditionOf(text);
  if (newCond && newCond !== state.world.locations[locName].condition) {
    state.world.locations[locName].condition = newCond;
    if (state.emergence.config.LocationAutoUpdate === "Enabled") this.process
  }
}
}

// A location's felt atmosphere, derived from its condition and current world
// tension rather than stored separately - always in sync, no extra state.
static locationAtmosphere(loc) {
  if (loc.condition === "On Fire" || loc.condition === "Destroyed") return "Cha
  if (loc.condition === "Locked Down") return "Tense";
  if (state.emergence.worldTension > 50) return "On Edge";
  if (state.emergence.worldTension > 25) return "Uneasy";
  return "Calm";
}

static processLocationCards(forceCreate = false, locNameOverride = null) {

```



```

if (state.emergence.config.LocationCards !== "Enabled") return;
if (typeof storyCards === 'undefined' || !Array.isArray(storyCards)) return;
const locName = locNameOverride || state.emergence.currentLocation;
if (!locName || locName === "Unknown Location") return;

if (!state.world.locations[locName]) {
    state.world.locations[locName] = { condition: "Intact" };
}

const loc = state.world.locations[locName];
let entryText = `[LOCATION: ${locName}]\nA dynamic environment mapped by EMER
if (loc.echo) entryText += `\nEcho: ${loc.echo}`;

let card = storyCards.find(c => c && c.category === "Locations" && c.title ==

if (!card && forceCreate) {
    const newCard = this.createStoryCard(locName.toLowerCase(), entryText, "Loc
    if (newCard) newCard.category = "Locations";
} else if (card && state.emergence.config.LocationAutoUpdate === "Enabled") {
    card.entry = entryText;
}
}

static initializeNPC(m) {
    const biases = ["Paranoid", "Deflective", "Prideful", "Impulsive", "Skeptical
    const attachments = ["Secure", "Anxious", "Avoidant", "Disorganized"];
    const goals = ["Survive the day", "Gain leverage", "Hide their past", "Protec
    const secrets = ["Betrayed an ally", "Owes a massive debt", "Harbors forbidde

state.world.npcs[m] = {
    trust: 50, grudge: 0, stress: 10, egoReserve: 100, attraction: 0,
    cognitiveBias: biases[Math.floor(Math.random() * biases.length)],
    attachmentStyle: attachments[Math.floor(Math.random() * attachments.length)
    hiddenGoal: goals[Math.floor(Math.random() * goals.length)],
    secret: secrets[Math.floor(Math.random() * secrets.length)],
    coreBelief: "Unsure of their place in the world.",
    threatState: "Composed", mood: "Neutral", outerMask: "Guarded.", innerMind:
    memories: [], coreMemory: "",
    undercurrents: {},
    reflections: [],
    lastSeenLocation: "", lastSeenTurn: 0, secureStreak: 0
};
this.adoptExistingCard(m);
}

// If the scenario author already made a Story Card for this character
// before this script ever ran, "adopt" it into the auto-update system
// instead of ignoring it and creating a separate one. Adoption ONLY marks

```

```

// the card so its EMERGENCE STATE block gets appended/refreshed going
// forward (through the exact same updateCharacterCards() path a /card-made
// sheet uses, which already preserves everything above that marker) - it
// never rewrites, replaces, or touches a single word of the author's
// existing lore. AI-authored profile-fill (which invents Race/Background/
// etc.) is a completely separate path that only ever runs for cards this
// script created itself via /card - an adopted card never goes through it,
// so nothing gets invented on top of what the author already wrote.
static adoptExistingCard(name) {
  if (typeof storyCards === 'undefined' || !Array.isArray(storyCards)) return;
  const card = storyCards.find(c => c && c.category !== "Characters" && c.type
    c.title && c.title.toLowerCase() === name.toLowerCase());
  if (card) {
    card.category = "Characters";
  }
}

// Finds capitalized words and flags whether each occurred at the START of a
// sentence (where English capitalizes ANY word) or genuinely mid-sentence
// (a much stronger signal of a real proper noun).
static scanCapitalizedWords(text) {
  const results = [];
  const re = /\b([A-Z][a-z]{2,10})\b/g;
  let m;
  while ((m = re.exec(text)) !== null) {
    let i = m.index - 1;
    let crossedQuote = false;
    while (i >= 0 && /\s"'`u2018u2019u201Cu201D/.test(text[i])) {
      if (/["'`u2018u2019u201Cu201D]/.test(text[i])) crossedQuote = true;
      i--;
    }
    const prevChar = i >= 0 ? text[i] : null;
    // The first word inside an opening quote is treated as sentence-initial
    // too, even if a comma (not a full stop) introduces it - "Marcus said,
    // "Don't move."" opens a genuinely new sentence at "Don't", but the
    // character right before the quote is just a comma. Without this, the
    // first word of nearly any line of dialogue was being read as a
    // confirmed mid-sentence proper noun and instantly promoted to an NPC.
    const sentenceInitial = (prevChar === null) || /[.!?]/.test(prevChar) || cr
    results.push({ word: m[1], sentenceInitial: sentenceInitial });
  }
  return results;
}

static discoverCharacters(text) {
  if (!text || state.emergence.config.NPCBrainSystem !== "Enabled") return;
  // Common words that get capitalized only because they start a sentence, not

```

```

// because they're names. This list is the first line of defense; the
// mid-sentence check below is the second.
const ignore = [
  "Dan", "Eve", "Door", "Shadows", "Table", "Sun", "Moon", "City", "Street",
  "I", "You", "He", "She", "It", "They", "We", "His", "Her", "Him", "Them", "
  "Its", "My", "Our", "Your", "Mine", "Yours", "Ours", "Theirs", "Myself", "Y
  "Himself", "Herself", "Itself", "Themselves", "Ourselves",
  "The", "A", "An", "This", "That", "These", "Those", "Some", "Any", "No", "N
  "Each", "Either", "Neither", "All", "Both", "Few", "Many", "Much", "Several
  "Who", "What", "When", "Where", "Why", "How", "Which", "Whose",
  "And", "But", "Or", "So", "Yet", "For", "Nor", "Because", "Although", "Thou
  "Since", "Unless", "While", "Whereas", "If", "Even",
  "With", "Without", "From", "Into", "Onto", "Upon", "Over", "Under", "Throug
  "Around", "Behind", "Beside", "Between", "Among", "About", "Above", "Below"
  "Near", "Toward", "Towards", "Against", "Across", "Along", "Beyond", "Withi
  "Before", "After", "During", "Until", "Despite", "Inside", "Outside",
  "Then", "Now", "Here", "There", "Suddenly", "Meanwhile", "Finally", "Eventu
  "Perhaps", "Maybe", "Still", "Also", "Instead", "However", "Therefore", "Ot
  "Indeed", "Certainly", "Obviously", "Clearly", "Slowly", "Quickly", "Quietl
  "Silently", "Immediately", "Soon", "Later", "Again", "Already", "Almost", "
  "Only", "Rather", "Nearly", "Yes", "No", "Well", "Okay", "Ah", "Oh", "Hey",
  "Look", "Listen", "Morning", "Night", "Evening", "Afternoon", "Today", "Tom
];

const found = this.scanCapitalizedWords(text);
if (!found.length) return;

if (!state.emergence.nameCandidates) state.emergence.nameCandidates = {};
const cand = state.emergence.nameCandidates;

found.forEach(f => {
  const m = f.word;
  if (ignore.includes(m) || state.world.npcs[m] || state.world.locations[m])
    if (Object.keys(state.world.npcs).length >= 25) return;

  if (!cand[m]) cand[m] = { count: 0, mid: false };
  cand[m].count++;
  if (!f.sentenceInitial) cand[m].mid = true;

  // Promote immediately on a genuine mid-sentence sighting (a strong signal)
  // or as a last resort after several sentence-initial mentions, so a real
  // name mentioned only at sentence starts is still eventually caught.
  if (cand[m].mid || cand[m].count >= 5) {
    this.initializeNPC(m);
    delete cand[m];
  }
});

```

```

// Keep the candidate pool bounded so it can't grow unchecked over a long gam
const keys = Object.keys(cands);
if (keys.length > 60) {
  keys.sort((a, b) => cands[a].count - cands[b].count);
  for (let i = 0; i < keys.length - 60; i++) delete cands[keys[i]];
}
}

// Which known NPCs (whole-word match) appear in the recent story text. Scans a
// bounded trailing window so this stays cheap even in a very long adventure.
static getSceneNames(text) {
  if (!text) return [];
  const window = text.length > 1500 ? text.slice(-1500) : text;
  const names = Object.keys(state.world.npcs);
  const found = [];
  for (let i = 0; i < names.length; i++) {
    const re = new RegExp(`\\b${this.escapeRegExp(names[i])}\\b`, 'i');
    if (re.test(window)) found.push(names[i]);
  }
  return found;
}

// Attachment style shapes stress response as much as cognitive bias does -
// an Avoidant NPC shuts down (Freeze) under the same pressure that sends an
// Anxious one seeking appeasement (Fawn) instead of fighting or fleeing.
// Threat states follow the real polyvagal hierarchy: Freeze (dorsal vagal
// shutdown) is reserved for the most extreme tier since it's more severe
// than the sympathetic fight/flight/fawn responses, not an equal alternative
// to them. Fawn stays available to Anxious-attachment NPCs even at extreme
// stress, since real research frames fawning as their persistent go-to
// strategy across the threat range, not something that stops working under
// real pressure and gets replaced by fight.
static getThreatState(stress, bias, attachmentStyle) {
  if (stress < 30) return "Composed";
  if (stress > 80) {
    if (attachmentStyle === "Anxious") return "Fawn";
    if (bias === "Paranoid" || bias === "Skeptical" || attachmentStyle === "Avo
    return "Fight";
  }
  if (stress > 60) {
    if (attachmentStyle === "Anxious") return "Fawn";
    return "Flight";
  }
  return "Fawn";
}

```

```

// How fast an NPC's trust actually rises in response to being treated well -
// a Skeptical or Paranoid NPC makes you earn it; an Impulsive one warms fast.
static trustGainMultiplier(bias) {
  const map = { "Skeptical": 0.6, "Paranoid": 0.7, "Prideful": 0.85, "Impulsive": 0.95 };
  return map[bias] !== undefined ? map[bias] : 1;
}

// Pushback isn't just AutonomyLevel in a vacuum - an NPC who already resents
// you escalates faster (piling on), while one who's earned deep trust with
// you gives a bit more benefit of the doubt for the same forceful moment.
static contextualPushbackMultiplier(npc) {
  let mult = this.autonomyMultiplier();
  if (npc.grudge > 60) mult *= 1.3;
  if (npc.trust > 70) mult *= 0.8;
  else if (npc.trust < 30) mult *= 1.15;
  return mult;
}

static simulateSelfReflection() {
  // Grudge cools slightly over time if nothing reinforces it - anger fades
  // even when trust still has to be actively rebuilt, so this only touches
  // grudge, and only by a small amount relative to how fast betrayal raises it
  // Ego reserve recovers the same way, but only while an NPC isn't actively
  // stressed - you can't regain your composure while still under pressure.
  // Undercurrents soften the same way too, for the same reason grudge does -
  // tension between two NPCs that nothing keeps reinforcing gradually eases.
  if (state.emergence.config.GrudgeTracking === "Enabled" || state.emergence.config.EgoReserveRecovery === "Enabled") {
    Object.keys(state.world.npcs).forEach(name => {
      const npc = state.world.npcs[name];
      if (state.emergence.config.GrudgeTracking === "Enabled" && npc.grudge > 0) {
        npc.grudge = this.clamp(npc.grudge - 2, 0, 100);
      }
      if (state.emergence.config.NPCBrainSystem === "Enabled" && npc.egoReserve < 100) {
        npc.egoReserve = this.clamp(npc.egoReserve + 3, 0, 100);
      }
    });
  }

  if (state.emergence.config.LivingWorldEngine === "Enabled") {
    Object.keys(state.world.npcs).forEach(name => {
      const npc = state.world.npcs[name];
      if (!npc.undercurrents) return;
      Object.keys(npc.undercurrents).forEach(target => {
        const w = npc.undercurrents[target];
        const next = w > 0 ? Math.max(0, w - 1) : Math.min(0, w + 1);
        if (next === 0) delete npc.undercurrents[target];
        else npc.undercurrents[target] = next;
      });
    });
  }
}

```

```

    });
  });
}

if (state.emergence.config.InnerSelfSystem !== "Enabled") return;
Object.keys(state.world.npcs).forEach(name => {
  let npc = state.world.npcs[name];
  if (npc.egoReserve < 30 || npc.stress > 80) {
    npc.hiddenGoal = "Prioritize immediate survival and escape.";
    npc.coreBelief = "The world is unsafe. I must protect myself.";
  } else if (npc.trust > 80) {
    npc.coreBelief = "The protagonist is a trusted ally and friend.";
  } else if (npc.grudge > 80 || npc.trust < 20) {
    npc.coreBelief = "The protagonist is a severe threat and must be dealt wi
  }
});
}

// WORD GETS AROUND
// Reputation quietly spreads between NPCs who are genuine confidants (a
// strong Undercurrent). If A trusts B, and B has a much harsher opinion of
// the player than A does, some of that rubs off on A - and vice versa. This
// only exists because Trust/Grudge and multi-target Undercurrents are both
// already tracked; it's the two systems talking to each other, not a new
// one bolted on. Bounded and slow by design - a ripple, not a copy.
static spreadReputation() {
  if (state.emergence.config.LivingWorldEngine !== "Enabled") return;
  const names = Object.keys(state.world.npcs);
  if (names.length < 2) return;
  names.forEach(a => {
    const npcA = state.world.npcs[a];
    if (!npcA.undercurrents) return;
    Object.keys(npcA.undercurrents).forEach(b => {
      if (npcA.undercurrents[b] < 50) return; // only close confidants gossip
      const npcB = state.world.npcs[b];
      if (!npcB) return;
      const grudgeDiff = npcB.grudge - npcA.grudge;
      if (grudgeDiff > 25) this.updateStat(npcA, 'grudge', 3);
      else if (grudgeDiff < -25) this.updateStat(npcA, 'grudge', -2);
      // A confidant vouching for you spreads too, not just one warning you off
      const trustDiff = npcB.trust - npcA.trust;
      if (trustDiff > 25) this.updateStat(npcA, 'trust', 2);
      else if (trustDiff < -25) this.updateStat(npcA, 'trust', -2);
    });
  });
}
}

```

```

// ROMANCE ENGINE
// Attraction (0-100, toward the player) maps to a discrete stage so the
// narrator - and the player, via /romance and /npc - sees a meaningful label
// instead of a raw number. RomancePacing scales how fast attraction climbs.
static getRelationshipStage(attraction) {
  if (attraction >= 80) return "In Love";
  if (attraction >= 60) return "Romantic";
  if (attraction >= 40) return "Flirtatious";
  if (attraction >= 20) return "Interested";
  return "Platonic";
}

static romancePacingMultiplier() {
  const map = { "Fast": 1.6, "Normal": 1, "Slow Burn": 0.5 };
  const v = map[state.emergence.config.RomancePacing];
  return v !== undefined ? v : 1;
}

// How jealousy actually reads on an NPC depends on their attachment style -
// pure flavor text, not a tracked object, so it costs nothing to vary.
static jealousyInnerMind(attachmentStyle) {
  const map = {
    "Secure": "Feels a quiet concern, but trusts it'll pass.",
    "Anxious": "Spiraling - certain this means they're about to be replaced.",
    "Avoidant": "Shutting down, already pulling back before it can hurt more.",
    "Disorganized": "Torn between wanting to lash out and wanting to disappear."
  };
  return map[attachmentStyle] || "Boiling with intense jealousy.";
}

// One-shot "act on this now" nudge for a relationship stage that just changed.
static buildRomanceNudge() {
  const queue = state.emergence.pendingRomanceStageChange;
  if (!Array.isArray(queue) || !queue.length) return "";
  const items = queue.slice();
  state.emergence.pendingRomanceStageChange = [];
  const lines = items.map(p => `${p.name}'s feelings toward the player have shi
  const plural = items.length > 1;
  return `${lines.join('. ')}$. Let ${plural ? 'these' : 'this'} show through be
}

// One-shot nudge for Earned Security, mirroring buildRomanceNudge - this is
// a real psychological milestone (an attachment style shifting because of
// how they've been treated), and it deserves the same chance to actually
// show up in the story as a romance stage change does, not just sit silently
// in the background data until someone happens to check /npc.
static buildSecurityNudge() {

```

```

const queue = state.emergence.pendingSecurityShift;
if (!Array.isArray(queue) || !queue.length) return "";
const names = queue.slice();
state.emergence.pendingSecurityShift = [];
if (names.length === 1) {
    return `${names[0]} has genuinely learned they can trust the player, after
}
return `${names.join(' and ')} have each genuinely learned they can trust the
}

// UNDERCURRENTS
// NPC-to-NPC relationship dynamics, inferred PASSIVELY from what's already
// happening in the story - reusing the same conjugation-aware trigger
// patterns that drive player-NPC trust/grudge. Nothing is scheduled, nothing
// is invented, and nothing asks the narrator to report back: if two known
// NPCs are mentioned together in a passage and a trigger fires, their
// Undercurrent shifts. That's the entire mechanism.
static updateUndercurrents(text) {
    if (!text || state.emergence.config.LivingWorldEngine !== "Enabled") return;
    const names = Object.keys(state.world.npcs);
    if (names.length < 2) return;

    const lower = text.toLowerCase();
    const p = this.getTriggerPatterns();
    let delta = 0;
    let directSignal = false;
    let memoryVerb = "";
    if (p.pushback.test(lower) || p.betrayal.test(lower)) { delta -= 15; directSi
    if (p.respect.test(lower)) { delta += 10; directSignal = true; memoryVerb = "
    // A romantic moment only counts as warmth BETWEEN the two co-mentioned NPCs
    // if the player doesn't appear to be the one involved - "Elena kisses you
    // while Marcus watches" is Marcus witnessing something between Elena and
    // the player (that's jealousy's job, handled separately in Output), not
    // warmth between Marcus and Elena themselves.
    if (p.romance.test(lower) && !/\byou(r|rs|rself)?\b/i.test(lower)) { delta +=
    // Shared danger bonds people, absent any direct conflict or warmth between
    // them specifically - a small, distinct signal from the ones above.
    if (!directSignal && p.tensionUp.test(lower)) { delta = 8; memoryVerb = "Got
    if (delta === 0) return;

    const mentioned = names.filter(n => new RegExp(`\\b${this.escapeRegExp(n)}\\b
    if (mentioned.length < 2) return;

    for (let i = 0; i < mentioned.length; i++) {
        for (let j = i + 1; j < mentioned.length; j++) {
            const a = mentioned[i], b = mentioned[j];
            this.nudgeUndercurrent(a, b, delta);

```



```

        this.nudgeUndercurrent(b, a, delta);
        // NPCs remember specific incidents with EACH OTHER too, not just with
        // the player - the same way Undercurrents already track that these
        // incidents happen.
        if (memoryVerb) {
            this.pushIncidentMemory(a, `${memoryVerb} ${b}.`);
            this.pushIncidentMemory(b, `${memoryVerb} ${a}.`);
        }
    }
}

static pushIncidentMemory(name, text) {
    const npc = state.world.npcs[name];
    if (!npc || !Array.isArray(npc.memories)) return;
    npc.memories.push(text);
    if (npc.memories.length > 3) npc.memories.shift();
}

// Each NPC can carry Undercurrents toward SEVERAL other characters at once,
// not just whoever they were most recently mentioned alongside - tension with
// one person doesn't get silently erased by a warm moment with someone else.
static nudgeUndercurrent(name, towardName, delta) {
    const npc = state.world.npcs[name];
    if (!npc) return;
    if (!npc.undercurrents || typeof npc.undercurrents !== 'object') npc.undercur
    const current = npc.undercurrents[towardName] || 0;
    const next = this.clamp(current + delta, -100, 100);
    if (next === 0) {
        delete npc.undercurrents[towardName];
    } else {
        npc.undercurrents[towardName] = next;
    }
}

// Picks the single strongest Undercurrent (by absolute warmth) an NPC
// currently carries, for compact display where only one can be shown.
static strongestUndercurrent(npc) {
    if (!npc || !npc.undercurrents) return null;
    const names = Object.keys(npc.undercurrents);
    if (!names.length) return null;
    let best = names[0];
    names.forEach(n => {
        if (Math.abs(npc.undercurrents[n]) > Math.abs(npc.undercurrents[best])) bes
    });
    return { target: best, warmth: npc.undercurrents[best] };
}

```

```

// COALITIONS
// Nothing here is tracked separately - it's pure read-only analysis over the
// Undercurrent graph that already exists. Three or more NPCs who all hold
// strong warmth with EACH OTHER (a closed triangle, not just each liking a
// fourth party) form a Coalition; three or more all holding strong mutual
// hostility form a Rivalry Bloc. Entirely derived, entirely stateless -
// nothing to seed, nothing to keep in sync, nothing that can drift from
// the underlying relationships.
static detectCoalitions() {
  const names = Object.keys(state.world.npcs);
  const threshold = 35;
  const warmPairs = new Set();
  const coldPairs = new Set();

  names.forEach(a => {
    const npc = state.world.npcs[a];
    if (!npc.undercurrents) return;
    Object.keys(npc.undercurrents).forEach(b => {
      if (!state.world.npcs[b]) return;
      const w = npc.undercurrents[b];
      const key = [a, b].sort().join('|');
      if (w >= threshold) warmPairs.add(key);
      if (w <= -threshold) coldPairs.add(key);
    });
  });

  const findTriangles = (pairSet) => {
    const cliques = [];
    const seen = {};
    pairSet.forEach(pairKey => {
      const [a, b] = pairKey.split('|');
      names.forEach(c => {
        if (c === a || c === b) return;
        const key1 = [a, c].sort().join('|');
        const key2 = [b, c].sort().join('|');
        if (pairSet.has(key1) && pairSet.has(key2)) {
          const clique = [a, b, c].sort();
          const cliqueKey = clique.join(',');
          if (!seen[cliqueKey]) {
            seen[cliqueKey] = true;
            cliques.push(clique);
          }
        }
      });
    });
    return cliques;
  };

```

```

};

return { coalitions: findTriangles(warmPairs), rivalryBlocs: findTriangles(co
}

// Location Echoes: the same formative-moment triggers that create a
// permanent NPC memory also tag WHERE it happened - stateless, reuses the
// existing currentLocation, nothing new to keep in sync.
static recordLocationEcho(text) {
  const locName = state.emergence.currentLocation;
  if (!locName || locName === "Unknown Location") return;
  if (!state.world.locations[locName]) return;
  state.world.locations[locName].echo = text;
  this.processLocationCards(false, locName);
}

// Sets a permanent Formative Memory, tags the current location with a
// matching Echo, and refreshes the character card immediately - the same
// three steps every pivotal-moment trigger needs, in one place instead of
// copied at each call site (four of them: betrayal, In Love, Earned
// Security, and a first crisis threat state).
static setFormativeMemory(npc, memoryText, echoText) {
  if (!npc) return;
  npc.coreMemory = memoryText;
  this.recordLocationEcho(echoText);
  this.updateCharacterCards();
}

// Attachment-style-flavored verb for surfacing an Undercurrent in prose - a
// stateless lookup, not a tracked lifecycle. Two NPCs with the same warmth
// score can still read very differently depending on how each one attaches.
static undercurrentVerb(attachmentStyle, warmth) {
  const positive = {
    "Secure": "has grown genuinely fond of", "Anxious": "has grown anxiously at
    "Avoidant": "has, despite themselves, grown close to", "Disorganized": "feels
  };
  const negative = {
    "Secure": "has a real, grounded tension with", "Anxious": "feels a gnawing
    "Avoidant": "has quietly shut out", "Disorganized": "swings between resentm
  };
  const table = warmth > 0 ? positive : negative;
  return table[attachmentStyle] || (warmth > 0 ? "has grown close to" : "has fr
}

// A character card was just created with blank [Define X] placeholders. Only t
// narrator model actually knows whether this name is an established character
// (and who they are) or a brand-new one, so this asks it to fill the sheet in

```

```

// rather than the script guessing. Retries up to 2 turns, then gives up quietly
static buildProfileNudge() {
  const queue = state.emergence.pendingProfileFill;
  if (!Array.isArray(queue) || !queue.length) return "";
  const pending = queue[0];
  pending.attempts = (pending.attempts || 0) + 1;
  if (pending.attempts > 6) {
    queue.shift();
    return this.buildProfileNudge(); // give up on this one, try the next queue
  }
  const name = pending.name;
  return `IMPORTANT - a character card for ${name} was just created and is still
    <EOS_PROFILE>\nNAME: ${name}\nRACE: species or ancestry\nSTRENGTH: power 1
  `
}

static applyProfileFill(data) {
  if (!data || !data.name) return;
  if (typeof storyCards === 'undefined' || !Array.isArray(storyCards)) return;
  const actualName = this.findNpcByName(data.name) || this.toTitleCase(data.name);
  const card = storyCards.find(c => c.title.toLowerCase() === actualName);
  if (!card) return;

  const val = (v, fallback) => (v && String(v).trim()) ? String(v).trim() : fallback;
  const template = `Name: ${actualName}\nRace: ${val(data.race, "Unspecified")}\n
  `;

  card.entry = template.substring(0, 1490);
  this.updateCharacterCards();
  const queue = state.emergence.pendingProfileFill;
  if (Array.isArray(queue)) {
    const idx = queue.findIndex(p => p.name.toLowerCase() === actualName);
    if (idx !== -1) queue.splice(idx, 1);
  }
}

// Generic hidden-tag extractor, used for both the profile-fill and reflection
// write-backs. Returns the cleaned text plus the raw block contents (or null).
static extractAndStripTag(text, tag) {
  const open = `<${tag}>`, close = `</${tag}>`;
  const start = text.indexOf(open);
  if (start === -1) return { text: text, block: null };
  const end = text.indexOf(close, start + open.length);
  if (end === -1) return { text: text.slice(0, start).trim(), block: null };
  const block = text.slice(start + open.length, end);
  const cleaned = (text.slice(0, start) + text.slice(end + close.length)).replace(open, close);
  return { text: cleaned, block: block };
}

```

```

static parseTagFields(block) {
  const out = {};
  String(block).split('\n').forEach(line => {
    const m = line.match(/^(([A-Za-z]+):\s*(.+)$/);
    if (m) out[m[1].toLowerCase()] = m[2].trim();
  });
  return out;
}

// REFLECTIONS (private per-NPC thoughts). Uses the same hidden-tag mechanism
// as profile-fill, rather than a leading-parenthetical convention, so the
// whole script has one consistent way of asking the narrator for hidden data.
static attemptAskReflection(turnCount, sceneNames) {
  if (state.emergence.config.ReflectionSystem !== "Enabled") return "";
  const book = state.emergence.reflectionBook;
  const interval = parseInt(state.emergence.config.ReflectionInterval, 10) || 6
  if (turnCount - (book.lastTurn || 0) < interval) return "";
  const chance = parseInt(state.emergence.config.ReflectionChance, 10);
  if (Math.random() * 100 >= (isNaN(chance) ? 50 : chance)) return "";
  const candidates = sceneNames.filter(n => state.world.npcs[n]);
  if (!candidates.length) return "";

  const name = candidates[Math.floor(Math.random() * candidates.length)];
  const npc = state.world.npcs[name];
  book.lastTurn = turnCount;
  let feelingContext = `${npc.threatState.toLowerCase()}, ${npc.cognitiveBias.t
const strongestUC = this.strongestUndercurrent(npc);
if (strongestUC && Math.abs(strongestUC.warmth) >= 30) {
  feelingContext += `, ${strongestUC.warmth > 0 ? "quietly warm toward" : "ca
}
return `After the story below, add ONE hidden private reflection for ${name}
}

static appendReflection(name, text) {
  if (!name || !text) return;
  const actualName = this.findNPCByName(name);
  if (!actualName) return;
  const npc = state.world.npcs[actualName];
  if (!Array.isArray(npc.reflections)) npc.reflections = [];

  const trimmedText = String(text).trim();
  if (!trimmedText) return;
  const norm = s => s.toLowerCase().replace(/^[a-z0-9 ]/g, '').trim();
  const nt = norm(trimmedText);
  const isDupe = npc.reflections.slice(-3).some(r => norm(r.text) === nt);
  if (isDupe) return;

```

```

    if (!state.emergence.reflectionBook.seq) state.emergence.reflectionBook.seq =
    state.emergence.reflectionBook.seq[actualName] = (state.emergence.reflectionB
    npc.reflections.push({ n: state.emergence.reflectionBook.seq[actualName], tex

let joined = npc.reflections.map(r => `${r.n}. ${r.text}`).join('\n');
while (joined.length > 1400 && npc.reflections.length > 1) {
    npc.reflections.shift();
    joined = npc.reflections.map(r => `${r.n}. ${r.text}`).join('\n');
}
this.syncReflectionCard(actualName);
}

// Writes/updates the NPC's private reflections card. Uses a non-matching key
// so AI Dungeon never injects it into story context.
static syncReflectionCard(name) {
    if (typeof storyCards === 'undefined' || !Array.isArray(storyCards)) return;
    const npc = state.world.npcs[name];
    if (!npc || !Array.isArray(npc.reflections) || !npc.reflections.length) retur

    const key = "eos-reflect:" + name.toLowerCase();
    const title = name + " - Private Reflections";
    const body = npc.reflections.map(r => `${r.n}. ${r.text}`).join('\n');

    let card = storyCards.find(c => c && c.keys === key);
    if (!card) {
        this.createStoryCard(key, body, "Custom", title, "Private reflections. Neve
    } else {
        card.title = title;
        card.entry = body;
    }
}

static getGenreDefinitions() {
    if (!this._genreDefs) {
        const raw = {
            "Superhero": ["cape", "mutant", "power", "hero", "villain", "lair"],
            "Sci-Fi": ["laser", "space", "ship", "orbit", "AI", "galaxy"],
            "Horror": ["blood", "dark", "shadow", "creepy", "monster", "terror", "vam
            "Fantasy": ["vampire", "werewolf", "magic", "city", "hidden", "demon", "s
            "Cyberpunk": ["neon", "cyber", "hack", "corp", "implant", "chrome"],
            "Modern": ["phone", "car", "street", "coffee", "apartment", "city", "scre
        };
        this._genreDefs = {};
        for (const g in raw) {
            this._genreDefs[g] = raw[g].map(kw => new RegExp(`\\b${this.escapeRegExp(
        }
    }
}

```

```

    return this._genreDefs;
}

static detectGenre(text) {
  if (!text) return;
  const window = text.length > 4000 ? text.slice(-4000) : text;
  const defs = this.getGenreDefinitions();
  let best = state.emergence.config.Genre;
  let high = 0;
  for (const g in defs) {
    let score = 0;
    defs[g].forEach(re => {
      re.lastIndex = 0;
      if (re.test(window)) score++;
    });
    state.emergence.genreScores[g] = (state.emergence.genreScores[g] || 0) + score;
    if (state.emergence.genreScores[g] > high) {
      high = state.emergence.genreScores[g];
      best = g;
    }
  }
  state.emergence.config.Genre = best;
}

// Runs a bounded amount of maintenance work per turn instead of letting
// multiple heavy passes stack up together. Undercurrents need no periodic
// slot at all since they're purely passive/reactive to text already seen.
static runPeriodicMaintenance(turnCount, text) {
  if (turnCount <= 3) {
    this.detectGenre(text);
    return;
  }
  if (turnCount % 10 === 0) {
    this.detectGenre(text);
    this.updateCharacterCards();
  } else if (turnCount % 7 === 0) {
    this.spreadReputation();
  } else if (turnCount % 5 === 0) {
    this.simulateSelfReflection();
  } else if (turnCount % 3 === 0) {
    this.processLocationCards(false);
  }
}

static buildDirective() {
  // Front Memory is documented by AI Dungeon as "always included in full" and
  // never truncated/reduced like Story Cards or Author's Note are - it's the

```

```

// one field the platform can't gracefully trim on its own. Kept terse.
let d = `[EOS | Genre:${state.emergence.config.Genre} Style:${state.emergence

let flags = [];
if (state.emergence.config.MatureContent === "18+ Unrestricted") flags.push("
if (state.emergence.config.RomanceEngine === "18+ Unrestricted") flags.push("
if (state.emergence.config.GraphicRealism === "Unfiltered") flags.push("visce
if (flags.length) d += `[Unrestricted: ${flags.join(", ")}]\n`;

if (state.emergence.config.PlayerTrauma === "Enabled" && state.playerInner.st
  d += `[Player is ${state.playerInner.condition}: reflect sensory distortion
}

const locStr = state.emergence.currentLocation;
if (state.world.locations[locStr]) {
  let locLine = `[Loc: ${locStr} (${state.world.locations[locStr].condition})
  if (state.emergence.worldTension > 50) locLine += `, tension ${state.emerge
  d += locLine + `]\n`;
} else if (state.emergence.worldTension > 50) {
  d += `[Tension: ${state.emergence.worldTension}%. Environment chaotic.]\n`;
}

if (state.emergence.activeThreads.length > 0 && state.emergence.config.Protag
  d += `[Gossip: ${state.emergence.activeThreads[0]}\n`;
}

d = d.trim();
const MAX_DIRECTIVE_LENGTH = 400;
if (d.length > MAX_DIRECTIVE_LENGTH) {
  d = d.substring(0, MAX_DIRECTIVE_LENGTH - 3) + "...";
}
return d;
}

// Compact per-NPC color note (threat/bias/undercurrent/romance/ego). Goes to
// Author's Note rather than Front Memory - deeper detail lives on their
// Story Card via /card instead, which AI Dungeon can also trim if tight.
static buildNPCNote() {
  if (state.emergence.config.NpcColorNotes === "Disabled") return "";
  const scene = state.emergence.sceneNames || [];
  let activeNPCs = scene.filter(n => state.world.npcs[n]);
  if (!activeNPCs.length) activeNPCs = Object.keys(state.world.npcs).slice(-1);
  if (!activeNPCs.length) return "";

  const name = activeNPCs[0];
  const n = state.world.npcs[name];
  let line = `[${name}: ${n.threatState}/${n.cognitiveBias}`;

```



```

const strongestUC = this.strongestUndercurrent(n);
if (strongestUC && Math.abs(strongestUC.warmth) >= 30) {
    line += `, ${this.undercurrentVerb(n.attachmentStyle, strongestUC.warmth)}
}
if (n.egoReserve < 30) line += `, ego depleted (may blurt secrets/panic)`;
line += `]`;

if (state.emergence.config.RomanceEngine !== "Disabled" && n.attraction >= 20
    line += ` [Feels ${this.getRelationshipStage(n.attraction)} toward the play
}

// A real gap since this NPC was last actually present is worth reacting
// to - people notice absence, even in a text adventure.
if (n.lastSeenTurn && (state.emergence.turnCount - n.lastSeenTurn) > 15) {
    line += ` [Hasn't crossed paths with the player in a while - may react to t
}

const quirks = state.emergence.config.PhysicalQuirks;
if (quirks === "High") {
    line += ` [Weave in micro-tells: clenched jaws, avoided eye contact, shallo
} else if (quirks === "Moderate") {
    line += ` [Light physical tells where natural.];`
}
if (state.emergence.config.PsychologicalRealism === "Raw Human") {
    line += ` [Let subconscious motivation leak through even when unspoken.];`
}

return line;
}
}

```